

SESIÓN 6 | MLOps & AI BI

LAB 6

Despliegue con Docker: MLOps-Capstone-Project

Asignatura	MLOps & AI BI
Sesión	6 — Deploying Machine Learning Applications
Modalidad	Individual
Entrega	Dockerfiles + docker-compose.yml + PDF respuestas

Prerrequisitos: LAB 3 (app-iris-ct) + LAB 5 (Hugging Face Hub)

Contexto y motivación

En los laboratorios anteriores construiste un servicio de inferencia con FastAPI (app-iris-ct), lo dotaste de Continuous Training y subiste el modelo y el dataset a Hugging Face Hub. Con eso tienes los componentes fundamentales de un sistema MLOps: un artefacto versionado en la nube y una API que lo consume. El problema es que ejecutar esa API con `uvicorn main:app --reload` en tu máquina no es despliegue real: depende de tu entorno local, de tu versión de Python y de que nadie modifique el directorio `models/`.

En esta actividad vas a containerizar la aplicación completa: el backend FastAPI que sirve predicciones y el frontend que ofrece interfaz al usuario. Al finalizar tendrás los dos servicios arrancando con un único comando, aislados del sistema operativo del host y reproducibles en cualquier máquina que tenga Docker instalado.

Repositorio de trabajo

```
https://github.com/loyola-masters/MLOps-Capstone-Project
```

Clónalo en local antes de empezar:

```
git clone https://github.com/loyola-masters/MLOps-Capstone-Project.git
cd MLOps-Capstone-Project
```

Estructura del proyecto

```
MLOps-Capstone-Project/
├── backend-iris/
│   ├── main.py           <- API FastAPI de predicción
│   ├── requirements.txt
│   └── Dockerfile        <- a desarrollar
├── frontend-iris/
│   ├── gradio_app.py     <- interfaz de usuario (Gradio)
│   ├── requirements.txt
│   └── Dockerfile        <- a desarrollar
└── docker-compose.yml    <- creado
```

Los dos Dockerfile son los ficheros que vas a escribir en esta actividad. El resto del código ya está implementado.

Objetivos de aprendizaje

Al terminar esta actividad serás capaz de:

1. Escribir un `Dockerfile` correcto para una aplicación Python (backend FastAPI y frontend Gradio).
2. Comprender el sistema de `capas` de una imagen Docker y optimizar el orden de instrucciones para aprovechar la caché de construcción.
3. Distinguir entre la red del host y la red interna de Docker, y configurar la comunicación entre contenedores mediante nombres de servicio.
4. Usar Docker Compose para definir, arrancar y detener un stack multi-servicio con un único comando.
5. Gestionar variables de entorno para configurar el comportamiento de los contenedores sin modificar el código.
6. Reflexionar sobre qué aporta la containerización al ciclo de vida MLOps respecto al despliegue directo.

Conceptos clave antes de empezar

Imagen vs. contenedor

Una imagen es una plantilla inmutable construida a partir de un Dockerfile. Un contenedor es una instancia en ejecución de esa imagen. Puedes tener diez contenedores distintos corriendo a partir de la misma imagen sin que se interfieran entre sí.

Capas y caché

Cada instrucción de un Dockerfile genera una capa. Docker cachea las capas: si una capa no ha cambiado, se reutiliza. Por eso el orden importa:

```
# CORRECTO: requirements se instalan antes que el código fuente
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .

# INCORRECTO: cualquier cambio en el código invalida la caché de pip
COPY . .
RUN pip install -r requirements.txt
```

Red interna de Docker Compose

Cuando dos servicios comparten el mismo docker-compose.yml, Docker crea automáticamente una red interna. Dentro de esa red, cada servicio es accesible por su nombre de servicio, no por localhost. Si el backend se llama backend en el Compose, el frontend lo alcanza en http://backend:8000, no en http://localhost:8000.

Paso 1 — Dockerfile del backend

El backend es una API FastAPI que carga el modelo Iris desde Hugging Face Hub y expone predicciones en POST /predict. Crea el fichero backend-iris/Dockerfile:

```
# Imagen base oficial de Python, versión slim para reducir tamaño
FROM python:3.11-slim

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar e instalar dependencias ANTES que el código fuente
# Así la capa de pip se cachea aunque cambie el código
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar el código de la aplicación
COPY . .

# Puerto que expone el contenedor (documentación, no mapeo real)
EXPOSE 8000

# Bind en 0.0.0.0 para que sea accesible desde fuera del contenedor
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Construye y prueba la imagen en solitario:

```
cd backend-iris

# Construir imagen
docker build -t iris-backend .
```

```
# Ejecutar contenedor
docker run -d \
  -p 8000:8000 \
  --name iris-backend \
  iris-backend

# Verificar que responde
curl http://localhost:8000/health
```

Paso 2 — Dockerfile del frontend

El frontend es una aplicación Gradio que recibe los inputs del usuario, llama al backend y muestra la predicción. El puerto estándar de Gradio es 7860.

Crea el fichero frontend-iris/Dockerfile:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860

# Variable de entorno con la URL del backend
# Se sobrescribirá en docker-compose.yml con el nombre de servicio
ENV BACKEND_URL=http://localhost:8000

CMD ["python", "gradio_app.py"]
```

Nota: Gradio debe lanzarse con server_name="0.0.0.0" para ser accesible desde fuera del contenedor. Asegúrate de incluir ese parámetro en interface.launch().

Paso 3 — Docker Compose

Docker Compose define todos los servicios, sus configuraciones y sus relaciones en un único fichero. Escríbelo en la raíz del repositorio:

```
version: "3.9"

services:

  backend:
    build: ./backend-iris
    container_name: iris-backend
    ports:
      - "8000:8000"
    environment:
      - HF_TOKEN=${HF_TOKEN} # token de HF pasado desde el host
    restart: unless-stopped

  frontend:
    build: ./frontend-iris
    container_name: iris-frontend
    ports:
      - "7860:7860"
    environment:
      # En la red interna el backend se llama 'backend', no 'localhost'
      - BACKEND_URL=http://backend:8000
    depends_on:
      - backend
    restart: unless-stopped
```

Para el token de Hugging Face, crea un fichero .env en la raíz (añade al .gitignore):

```
# .env
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXX
```

Arranca el stack completo:

```
# Construir imágenes y arrancar en segundo plano
docker compose up --build -d

# Ver logs en tiempo real
docker compose logs -f

# Detener y eliminar contenedores
docker compose down
```

Accede a:

- Backend (documentación interactiva): <http://localhost:8000/docs>
- Frontend (interfaz de usuario): <http://localhost:7860>

Ejercicios de la actividad

Ejercicio 1 — Backend en solitario

Construye y arranca únicamente el contenedor del backend. Usa curl o el navegador para verificar que:

- GET /health responde correctamente.
- POST /predict con un JSON de ejemplo devuelve una predicción válida.

Captura el log de arranque del contenedor (docker logs iris-backend) y el resultado de ambas llamadas.

Ejercicio 2 — Frontend en solitario

Construye y arranca el contenedor del frontend apuntando al backend del Ejercicio 1 (usa -e BACKEND_URL=http://host.docker.internal:8000 en Windows/Mac). Realiza una predicción desde la interfaz gráfica y captura una captura de pantalla del resultado.

Ejercicio 3 — Stack completo con Docker Compose

Detén los contenedores anteriores y arranca el stack completo. Comprueba:

- docker compose ps muestra ambos servicios en estado running.
- El frontend puede comunicarse con el backend usando el nombre de servicio backend.
- Una predicción end-to-end funciona correctamente.

Captura la salida de docker compose ps y una predicción completa.

Ejercicio 4 — Verificación automática end-to-end

Es habitual producir scripts de bash para la comprobación del funcionamiento correcto de las aplicaciones. Tras haber realizado correctamente los ejercicios anteriores, haz el chequeo automático. Para ello, si estás en Windows, abre Git Bash. Navega a la carpeta del script y proporcióname permisos de ejecución:

```
chmod +x verify_stack.sh
./verify_stack.sh
```

Salida esperada:

```
=====
Verificación del Stack — LAB 6
Backend : http://localhost:8000
Frontend: http://localhost:7860
=====

[INFO] Esperando a que el backend esté listo (timeout: 60s)...
[OK]   Backend listo en 0s — modelo: v1.0-base
[INFO] Enviando predicción de prueba a POST http://localhost:8000/predict...
[OK]   Predicción recibida → clase=0, especie=setosa
       Payload enviado:  {"sepal_length": 5.1, ...}
       Respuesta completa: {"prediction":0,"species":"setosa","confidence":0.98}
[OK]   Frontend disponible (HTTP 200).

=====
STACK OK
=====
```

PREGUNTAS

Pregunta 1

En el CMD del Dockerfile del backend se usa `--host 0.0.0.0`. Si en vez de eso arrancas con `--host 127.0.0.1` (o sin especificar host, que es el valor por defecto de Uvicorn), el contenedor arranca sin errores pero no es posible acceder a él desde el host. ¿Por qué? ¿Qué significa 0.0.0.0 en el contexto de red de un contenedor y por qué es necesario para que el mapeo de puertos funcione?

Pregunta 2

En el `docker-compose.yml`, el frontend recibe la variable de entorno `BACKEND_URL=http://backend:8000`. Si cambiaras esa URL a `http://localhost:8000`, el frontend arrancarían correctamente pero fallaría al intentar llamar al backend. Explica por qué `localhost` dentro de un contenedor no resuelve al backend. ¿Qué mecanismo usa Docker Compose para que backend sea un nombre DNS válido dentro de la red interna?

Pregunta 3

En los Dockerfile de ambos servicios, las instrucciones siguen este orden:

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

¿Qué ocurriría con los tiempos de construcción si se invirtiera el orden y se hiciera `COPY . .` antes del `RUN pip install`? Describe el mecanismo de caché de capas de Docker y en qué situaciones habituales del desarrollo (cambiar una línea en `main.py`, añadir una dependencia nueva) nota la diferencia.

Pregunta 4

El fichero `.env` con el token de Hugging Face (`HF_TOKEN`) se referencia en el `docker-compose.yml` pero se añade al `.gitignore`. Existen al menos tres formas alternativas de pasar un secreto a un contenedor en producción: variable de entorno del shell, Docker Secrets (modo Swarm) y servicios externos de gestión de secretos (AWS Secrets Manager, HashiCorp Vault, etc.). Describe brevemente cada opción, identifica el riesgo principal de la solución con `.env` que usas en esta actividad y explica en qué tipo de entorno de producción elegirías cada alternativa.

EJERCICIO DE PROGRAMACIÓN

Health check y arranque ordenado

Idea central

El `depends_on` de Docker Compose garantiza que el servicio backend se haya iniciado antes que el frontend, pero no que la API esté lista para recibir peticiones. Si el backend tarda varios segundos en cargar el modelo de Hugging Face Hub, el frontend puede arrancar e intentar conectarse antes de que el backend esté operativo, produciendo errores en el inicio.

Fase de razonamiento previo

Considera los siguientes tres escenarios y razona cómo afecta el tiempo de carga:

- **Escenario 1:** modelo pequeño cargado localmente en menos de 1 segundo.
- **Escenario 2:** modelo descargado desde Hugging Face Hub al arrancar (latencia de red variable).
- **Escenario 3:** modelo grande que requiere compilación o warmup (decenas de segundos).

A partir de ese razonamiento, diseña en pseudocódigo cómo modificarías el `docker-compose.yml` y el backend para garantizar que el frontend solo intente conectarse cuando el backend esté realmente listo.

Funcionalidad a implementar

Extiende el proyecto con las siguientes modificaciones:

1. Health check en el Dockerfile del backend

```
HEALTHCHECK --interval=10s --timeout=5s --start-period=30s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1
```

2. Arranque ordenado en `docker-compose.yml`

Sustituye el `depends_on` simple por una condición que espere a que el backend supere su health check antes de arrancar el frontend:

```
depends_on:
  backend:
    condition: service_healthy
```

3. Endpoint `/health` robusto en el backend

El health check llama a `GET /health`. Asegúrate de que ese endpoint devuelva HTTP 200 solo cuando el modelo esté cargado en memoria, no simplemente cuando el proceso haya arrancado. Si el modelo aún no está listo, debe devolver HTTP 503.

Respuesta cuando el modelo está listo:

```
{
  "status": "ok",
  "model_loaded": true,
  "model_version": "v1.0-base",
  "uptime_seconds": 42
}
```

Respuesta cuando el modelo aún no está listo:

```
{
  "status": "not_ready",
  "model_loaded": false,
  "message": "Cargando modelo desde Hugging Face Hub..."
}
```

ENTREGABLES

1. backend-iris/Dockerfile funcional con HEALTHCHECK incluido.
2. frontend-iris/Dockerfile funcional.
3. docker-compose.yml con depends_on usando condition: service_healthy.
4. verify_stack.sh comprobado y muestra del output correcto.
5. Captura de pantalla o log de docker compose up --build mostrando ambos servicios en estado running (healthy).
6. Captura de una predicción end-to-end realizada desde el frontend.
7. Documento PDF (máx. 2 páginas) con las respuestas a las PREGUNTAS de reflexión y la fase de razonamiento previo del ejercicio de programación.

Referencias

- Docker Documentation — Dockerfile reference: <https://docs.docker.com/engine/reference/builder/>
- Docker Compose — Getting started: <https://docs.docker.com/compose/gettingstarted/>
- FastAPI — Deployment with Docker: <https://fastapi.tiangolo.com/deployment/docker/>
- Docker HEALTHCHECK: <https://docs.docker.com/engine/reference/builder/#healthcheck>
- Hugging Face Hub — Environment variables: https://huggingface.co/docs/huggingface_hub/
- Google MLOps: Continuous delivery and automation pipelines in ML:
<https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>